

Lab - 1:

Experiment No: 1

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
 - o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
 - o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
 - o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
 - o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

1) Python program that uses the hashlib library to create the hash of a given string:

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()

    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))

    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()

    # Return the hash string
    return hash_string

# Get input from the user
input_string = input("Enter your name: ")

# Generate hash
hash_result = create_hash(input_string)

print("Name:", input_string)
print("Hash:", hash_result)
```

Enter your name: Jatin Rohra

Name: Jatin Rohra

Hash: 9c91ee1747508f8cfc91b944a765759eb8e8f25b092987d61861c59ca98e4ae3

2) Program 2: Program to generate required target hash with input string and nonce

```
import hashlib

# Get user input
input_string = input("Enter your name: ")
nonce = input("Enter the nonce: ")
```

```

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Name:", input_string)
print("Nonce:", nonce)
print("Hash Code:", hash_code)
Enter your name: Jatin Rohra
Enter the nonce: 123
Name: Jatin Rohra
Nonce: 123
Hash Code:
9dc450d6f5428815a84071d02ec489c0aadae43ddd41ea4a69ffb8784c3b92f2

```

3)Python code for Solving Puzzle for leading zeros with expected nonce and given string

```

import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)

        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

```

```

        # Check if the hash starts with required zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce

    nonce += 1

# Get user input
input_string = input("Enter the input string: ")
num_zeros = int(input("Enter the number of leading zeros: "))

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the result
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)

```

Enter the input string: Jatin Rohra

Enter the number of leading zeros: 3

Hash: 000e826103542b2dd030171e736d187a2b0356e21519620bc34a768e9e3718bc

Input String: Jatin Rohra

Leading Zeros: 3

Expected Nonce: 1134

Program 4: Generating Merkle Tree for given set of Transactions

```

import hashlib

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    if len(transactions) == 1:
        return transactions[0]

```

```

# Construction of the Merkle Tree
while len(transactions) > 1:
    if len(transactions) % 2 != 0:
        transactions.append(transactions[-1])

    new_transactions = []

    for i in range(0, len(transactions), 2):
        combined = transactions[i] + transactions[i + 1]
        hash_combined = hashlib.sha256(
            combined.encode('utf-8')
        ).hexdigest()

        new_transactions.append(hash_combined)

    transactions = new_transactions

    return transactions[0]

# Get transactions from the user
num_transactions = int(input("Enter the number of transactions: "))

transactions = []

for i in range(num_transactions):
    transaction = input(f"Enter Transaction {i + 1}: ")
    transactions.append(transaction)

# Build the Merkle Tree
merkle_root = build_merkle_tree(transactions)

# Print the result
print("\nTransactions:", transactions)
print("Merkle Root:", merkle_root)
Enter the number of transactions: 3
Enter Transaction 1: Jatin Rohra
Enter Transaction 2: Test
Enter Transaction 3: foo

```

Transactions: ['Jatin Rohra', 'Test', 'foo', 'foo']

Merkle Root:

a154f15fff77f98b3d84547862b7bf1a43e68e4cbc3e77b44b692e8b28ed2943

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.